Machine-Level Programming III: Procedures

CSE 238/2038/2138: Systems Programming

Instructor:
Fatma CORUT ERGİN

Slides adapted from Bryant & O'Hallaron's slides

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ________,%rdi
  4004b0: jle 4004cc
  4004b2: add ________,________
  4004b4: mov $0x0,%rdx
  4004b9: cmp ________,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc ________
  4004c1: cmp %rdi,________
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retg
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
  # On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add ________,________
  4004b4: mov $0x0,%rdx
  4004b9: cmp ________,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc ________
  4004c1: cmp %rdi,________
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp ________,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc ________
  4004c1: cmp %rdi,________
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retg
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp __%rdi__,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc ________
  4004c1: cmp %rdi,________
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp __%rdi__,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc __%rdx__
  4004c1: cmp %rdi,________
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retg
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp __%rdi__,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc __%rdx__
  4004c1: cmp %rdi,__%rdx__
  4004c3: jl __________
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
  # On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp __%rdi__,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc __%rdx__
  4004c1: cmp %rdi,__%rdx__
  4004c3: jl __4004bd__
  4004c5: dec ________
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```

Example 1:

Consider the following C function and its x86-64 assembly code. Fill in the missing parts of the C code to get a program equivalent to the given assembly code.

```
long foo (long n) {
  long a, i;
  a = 0;
  for (; n > 1; n--) {
     a = a + n;
     for (i = 0; i < n; i++)
        a = a + i;
  }
  return a;
}
```

```
# On entry %rdi = n
0000000004004a8 <foo>:
  4004a8: mov $0x0,%rax
  4004ad: cmp ___$01__,%rdi
  4004b0: jle 4004cc
  4004b2: add __%rdi__,__%rax__
  4004b4: mov $0x0,%rdx
  4004b9: cmp __%rdi__,%rdx
  4004bb: jge 4004c5
  4004bd: add %rdx,%rax
  4004bf: inc __%rdx__
  4004c1: cmp %rdi,__%rdx__
  4004c3: jl __4004bd__
  4004c5: dec __%rdi__
  4004c7: cmp $0x1,%rdi
  4004ca: jg 4004b2
  4004cc: retq
```

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case _:
      case _:
            result = ______
            break;
      case _:
            x = ______
       /* Fall through */
      case _:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

```
0x5a0:   0xf6f
0x5a8:   0xf6a
0x5b0:   0xf7a
0x5b8:   0xf7f
0x5c0:   0xf84
0x5c8:   0xf6a
```

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case 1:
      case 5:
            result = ______
            break;
      case _:
            x = ______
       /* Fall through */
      case _:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case _:
            x = ______
       /* Fall through */
      case _:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = ______
       /* Fall through */
      case _:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case _:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case 2:
            result = ______
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case _:
            result = ______
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case 2:
            result = x + y;
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case 4:
            result = ______
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case 2:
            result = x + y;
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

0x5a0:	0xf6f
0x5a8:	0xf6a
0x5b0:	0xf7a
0x5b8:	0xf7f
0x5c0:	0xf84
0x5c8:	0xf6a

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case 4:
            result = x * 8;
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case 2:
            result = x + y;
            break;
      default:
            result = ______
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

```
0x5a0:   0xf6f
0x5a8:   0xf6a
0x5b0:   0xf7a
0x5b8:   0xf7f
0x5c0:   0xf84
0x5c8:   0xf6a
```

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

Example 2:

Consider the following C code and the assembly code that implements the branches of the switch statement of C code:

```
long fun(long x, long y){

    long result = 0;
    switch (y) {
      case 4:
            result = x * 8;
            break;
      case 1:
      case 5:
            result = y - 1;
            break;
      case 0:
            x = x * 7;
       /* Fall through */
      case 2:
            result = x + y;
            break;
      default:
            result = x & y;
    }
    return result;
}
```

```
f68: jmpq *0x5a0(,%rsi,8)
     f6a:   decq  %rsi
     f6d:   jmp   0xf8b
     f6f:   leaq  (,%rdi,8), %rsi
     f77:   subq  %rdi, %rsi
     f7a:   addq  %rdi, %rsi
     f7d:   jmp   0xf8b
     f7f:   andq  %rdi, %rsi
     f82:   jmp   0xf8b
     f84:   shlq  $3, %rdi
     f88:   movq  %rdi, %rsi
     f8b:   movq  %rsi, %rax
```

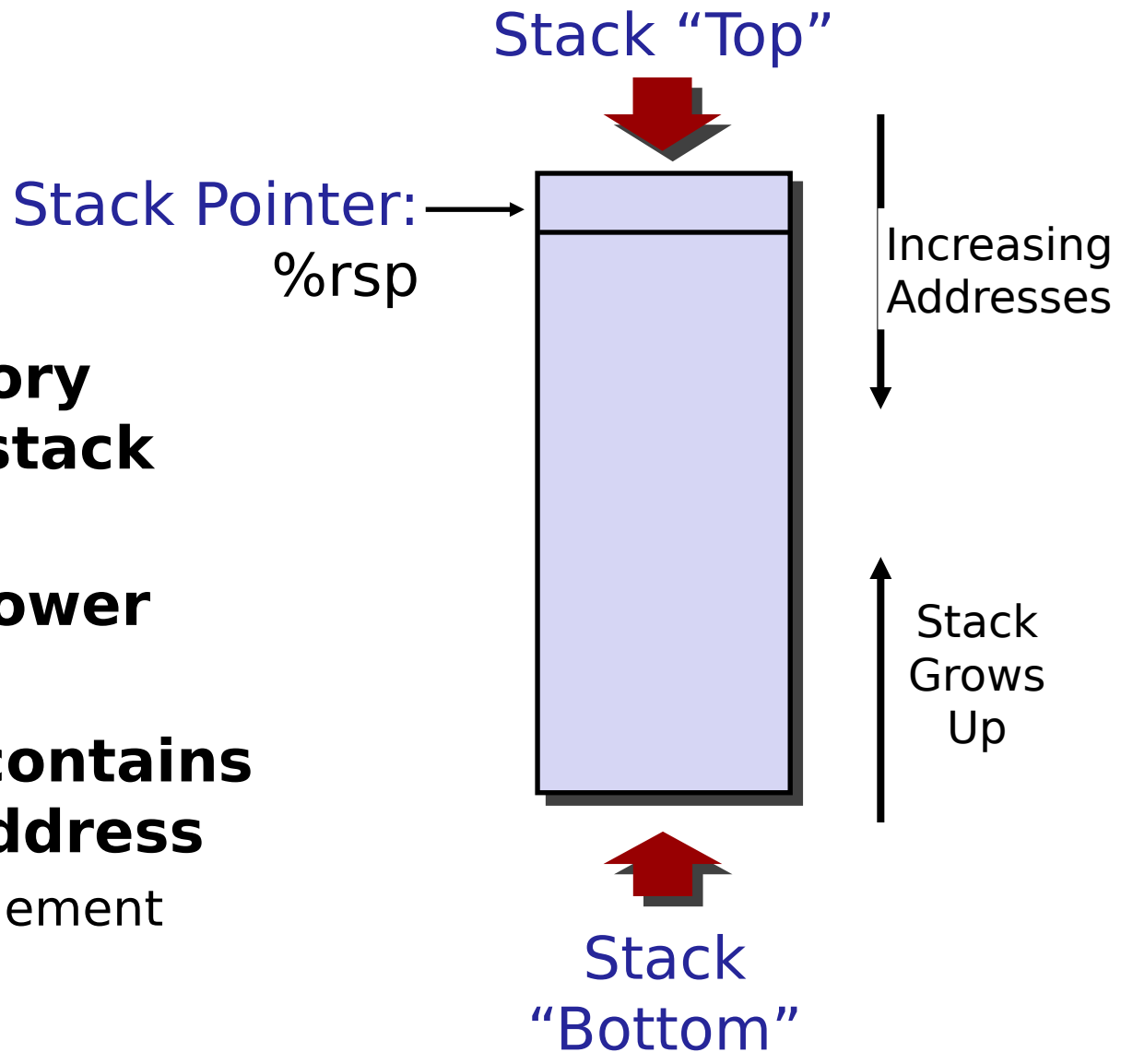
```
0x5a0:   0xf6f
0x5a8:   0xf6a
0x5b0:   0xf7a
0x5b8:   0xf7f
0x5c0:   0xf84
0x5c8:   0xf6a
```

Fill in the blank portions of the C code above to reproduce the function corresponding to the assembly code.

The jump table for the switch statement is given below. You can assume that the first entry in the jump table is for the case when `y=0`. Parameters `x` and `y` are passed in registers `%rdi`, and `%rsi`, respectively.

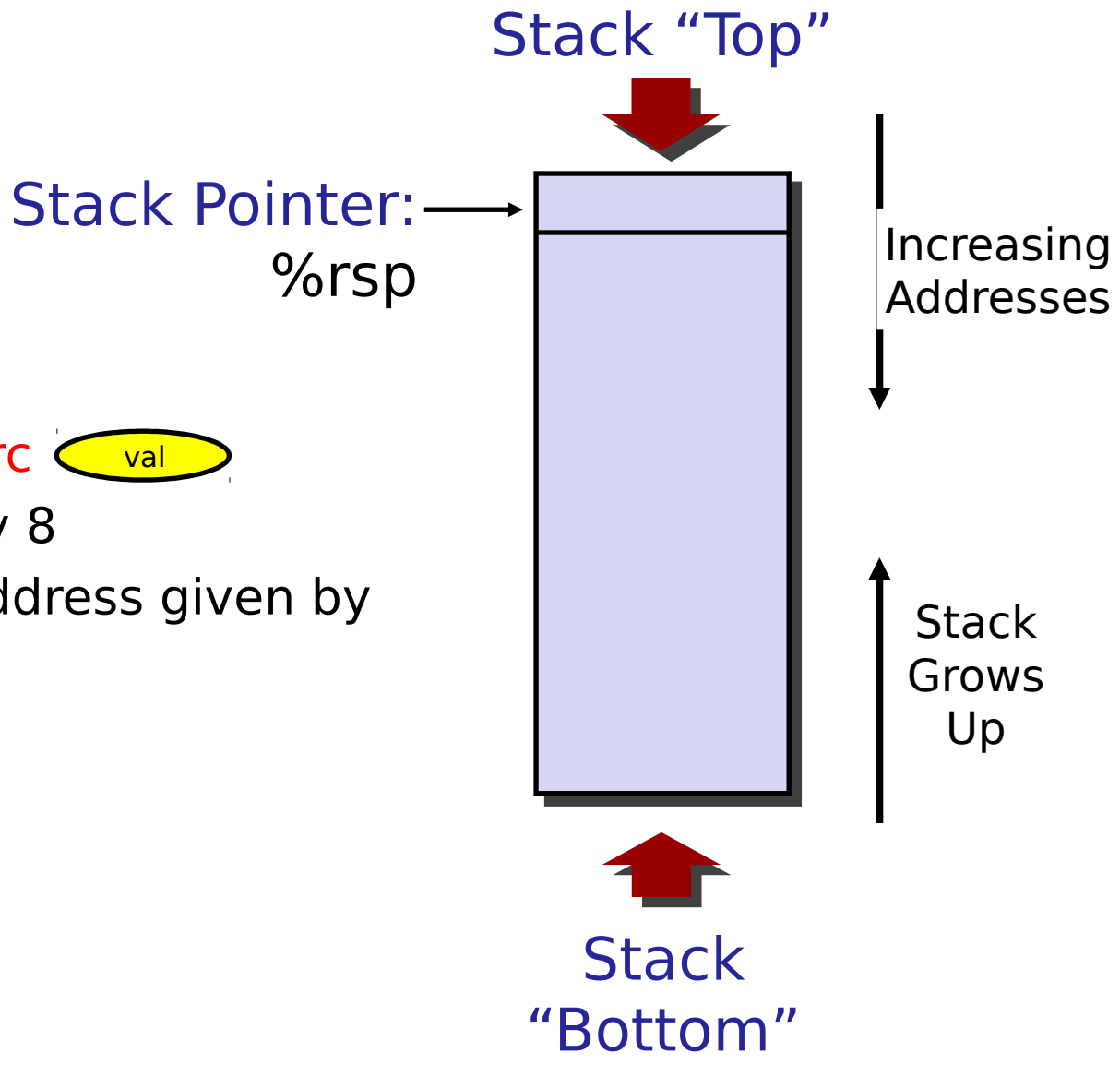
x86-64 Stack

- **Region of memory managed with stack discipline**
- **Grows toward lower addresses**
- **Register %rsp contains lowest stack address**
  - address of “top” element



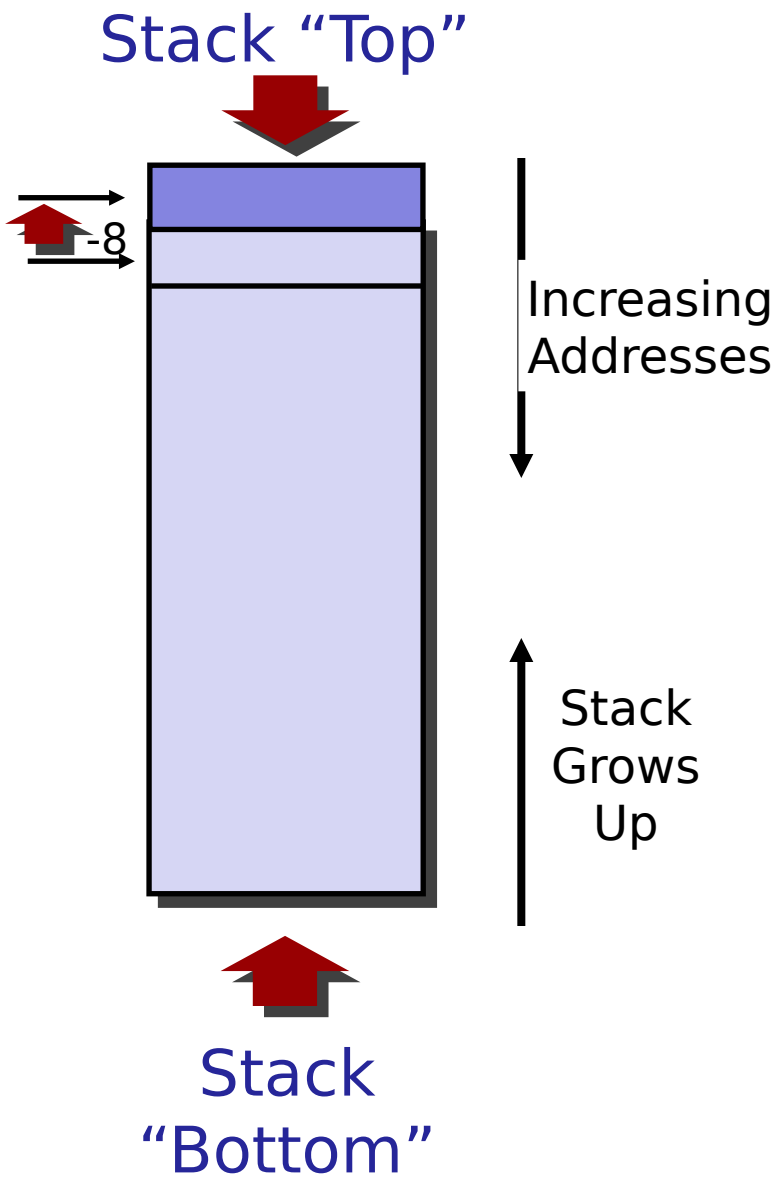
x86-64 Stack: Push

- **pushq Src**
  - Fetch operand at Src
  - Decrement %rsp by 8
  - Write operand at address given by %rsp



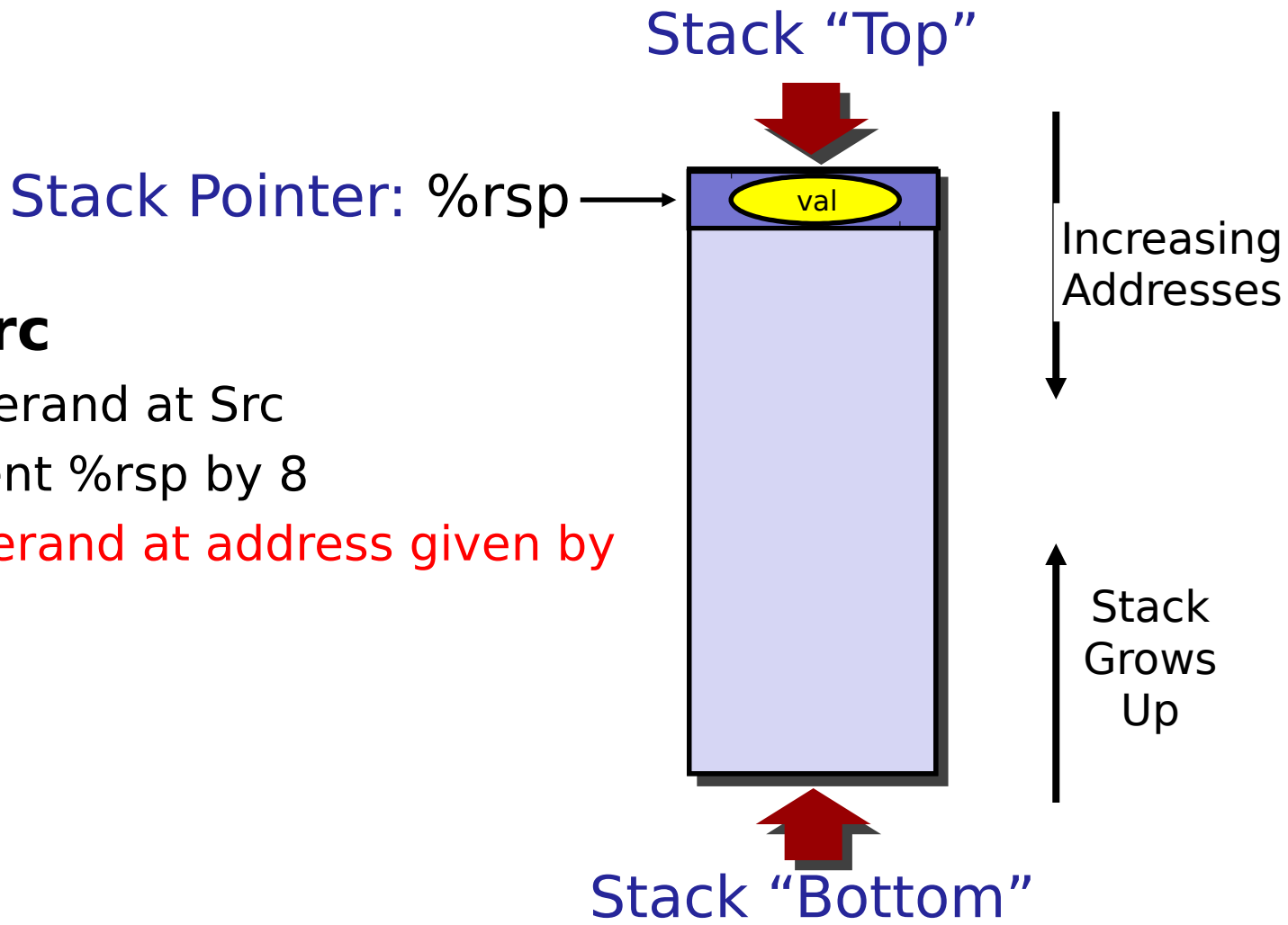
x86-64 Stack: Push

- **pushq Src**
  - Fetch operand at Src
  - Decrement %rsp by 8
  - Write operand at address given by %rsp

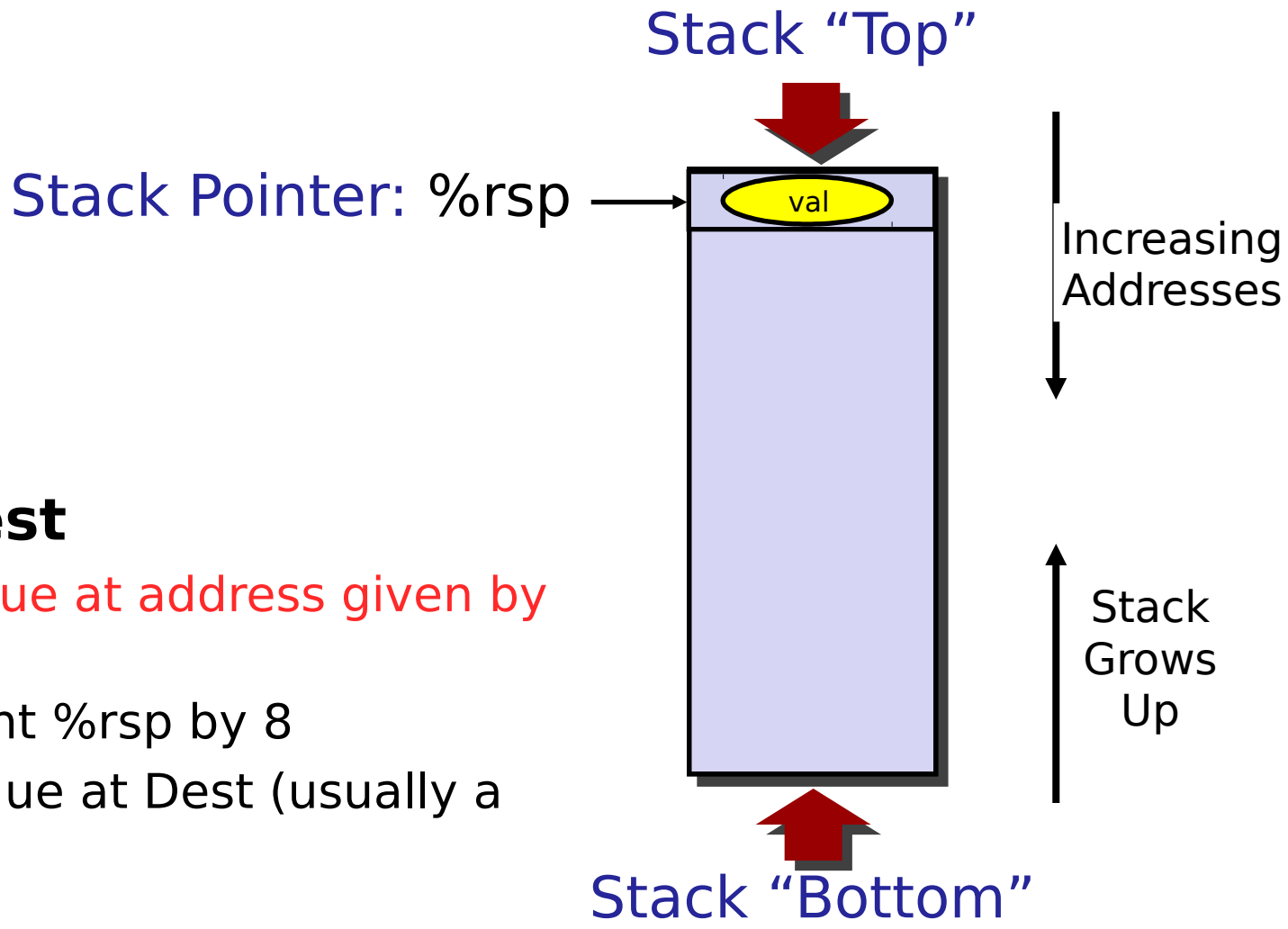


x86-64 Stack: Push

- **pushq Src**
  - Fetch operand at Src
  - Decrement %rsp by 8
  - Write operand at address given by %rsp

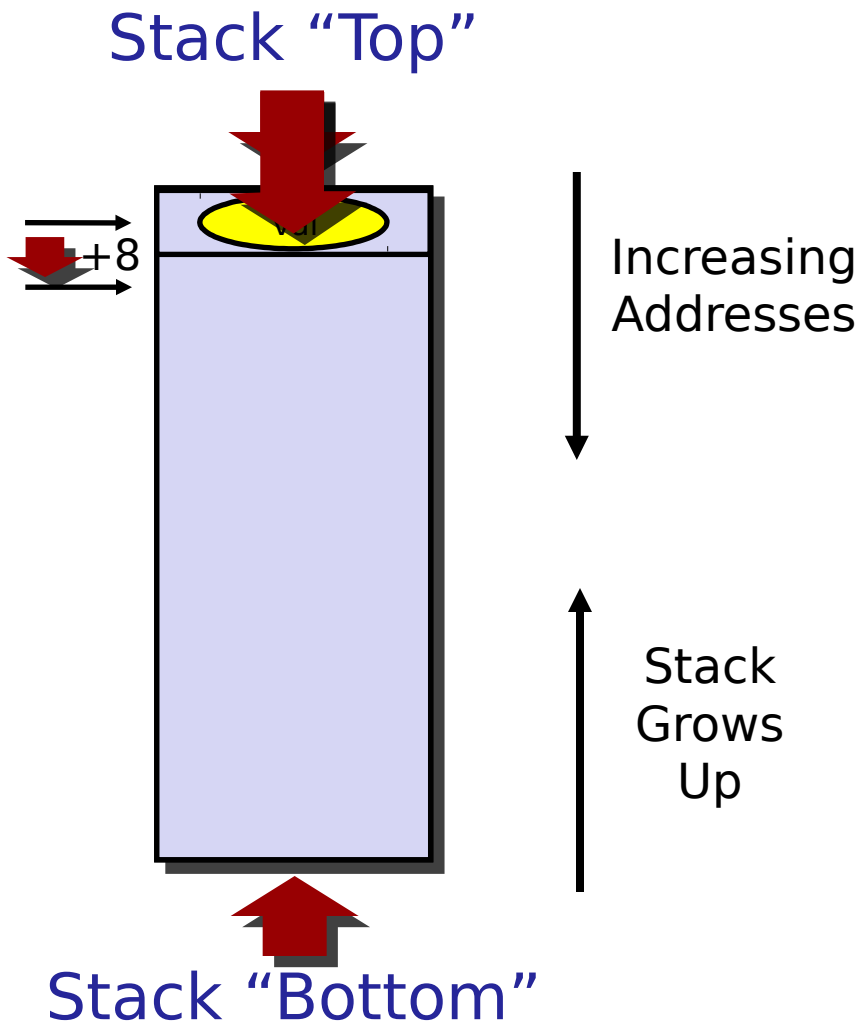


x86-64 Stack: Pop



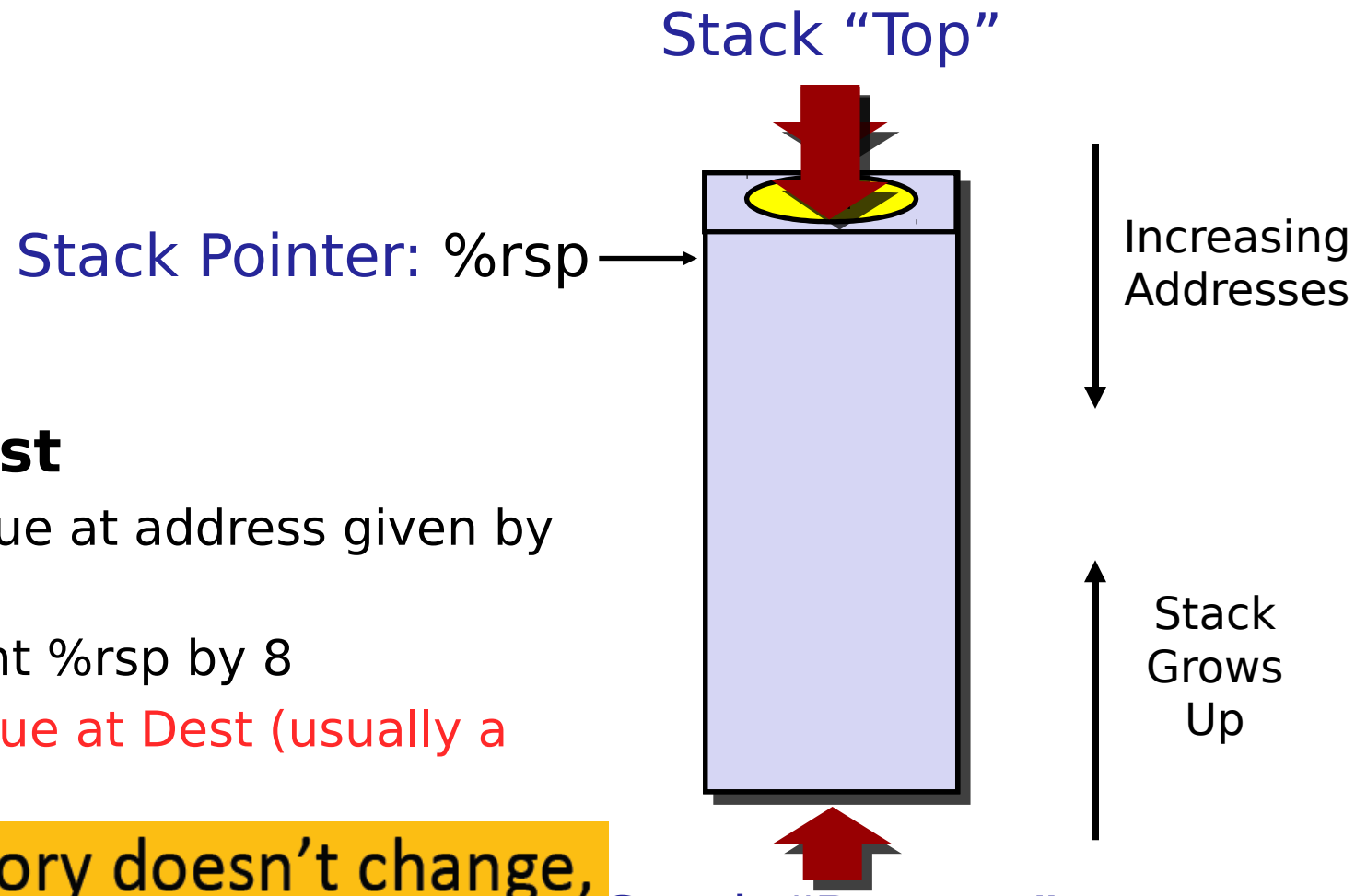
- **popq Dest**
  - Read value at address given by %rsp
  - Increment %rsp by 8
  - Store value at Dest (usually a register)

x86-64 Stack: Pop



- **popq Dest**
  - Read value at address given by %rsp
  - Increment %rsp by 8
  - Store value at Dest (usually a register)

x86-64 Stack: Pop



- **popq Dest**
  - Read value at address given by %rsp
  - Increment %rsp by 8
  - Store value at Dest (usually a register)

(The memory doesn't change, only the value of `%rsp`)

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){
  if ( _____________ )
    return ____;
  else{
    long a = _____________;
    unsigned long nx = a;
    long rv = rfun(nx);
    return ___ * rv;
  }
}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){
  if ( ____x==1____ )
    return ____;
  else{
    long a = _____________;
    unsigned long nx = a;
    long rv = rfun(nx);
    return ___ * rv;
  }
}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){
  if ( ____x==1____ )
    return _1__;
  else{
    long a = _____________;
    unsigned long nx = a;
    long rv = rfun(nx);
    return ___ * rv;
  }
}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){
  if ( ____x==1____ )
    return _1__;
  else{
    long a = ___x>>3_____;
    unsigned long nx = a;
    long rv = rfun(nx);
    return ___ * rv;
  }
}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

Example 3:

Consider the following C code and the assembly code for the same function:

```
rfun:
  pushq  %rbp
  movq   %rsp, %rbp
  movl   $1, %eax
  cmpq   $1, %rdi
  je     LBB0_3
LBB0_1:
  imulq  %rdi, %rax
  shrq   $3, %rdi
  cmpq   $1, %rdi
  jne    LBB0_1
LBB0_3:
  popq   %rbp
  retq
```

```
long rfun(unsigned long x){

  if ( ____x==1____ )

    return _1__;

  else{

    long a = ___x>>3_____;

    unsigned long nx = a;

    long rv = rfun(nx);

    return _x_ * rv;

  }

}
```

Fill in the missing parts of the C code to get a program equivalent to the generated assembly code.

Mechanisms in Procedures

- **Passing control**
  - To beginning of procedure code
  - Back to return point
- **Passing data**
  - Procedure arguments
  - Return value
- **Memory management**
  - Allocate during procedure execution
  - Deallocate upon return
- **Mechanisms all implemented with machine instructions**
- **x86-64 implementation of a procedure uses only those mechanisms required**

```
P(…) {
  •
  •
  y = Q(x);
  print(y)
  •
}
```

```
int Q(int i)
{
  int t = 3*i;
  int v[10];
  •
  •
  return v[t];
}
```

Code Examples

```
void multstore(long x, long y, long *dest)
{
    long t = mult2(x, y);
    *dest = t;
}
```

```
0000000000400540 <multstore>:
  400540: push   %rbx      # Save %rbx
  400541: mov    %rdx,%rbx    # Save dest
  400544: callq  400550 <mult2>  # mult2(x,y)
  400549: mov    %rax,(%rbx)  # Save at dest
  40054c: pop    %rbx      # Restore %rbx
  40054d: retq          # Return
```

```
long mult2(long a, long b)
{
  long s = a * b;
  return s;
}
```

```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax   # a
  400553:  imul   %rsi,%rax   # a * b
  400557:  retq         # Return
```

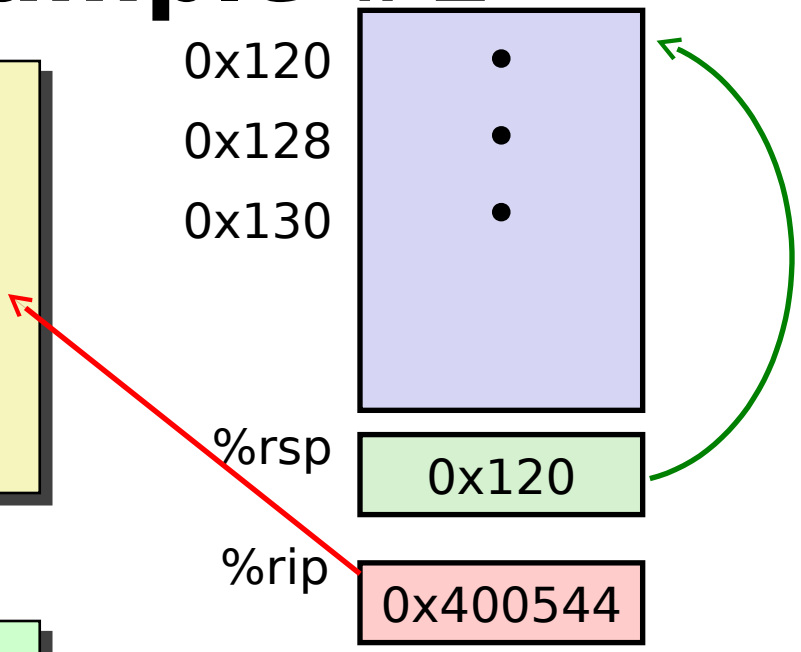
Procedure Control Flow

- **Use stack to support procedure call and return**
- **Procedure call:** `call label`
  - Push return address on stack
  - Jump to label
- **Return address:**
  - Address of the next instruction right after call
  - Example from disassembly
- **Procedure return:** `ret`
  - Pop address from stack
  - Jump to address

Control Flow Example #1

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

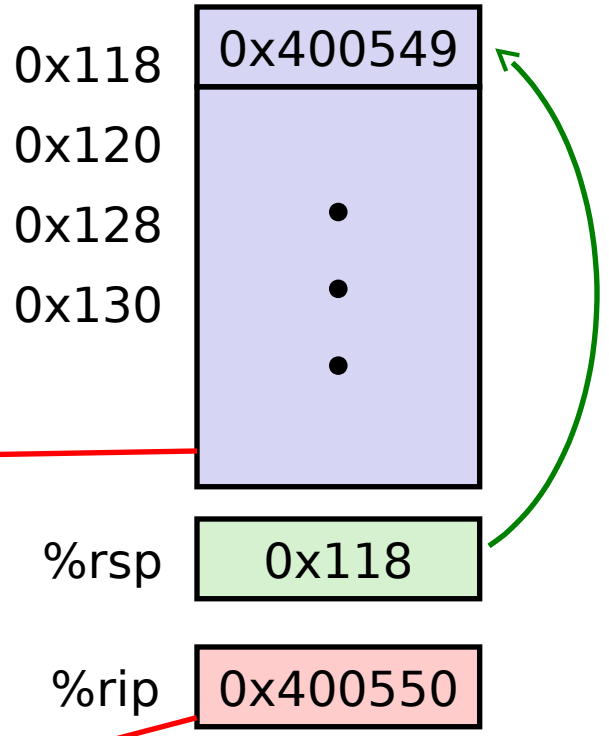
```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



Control Flow Example #2

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

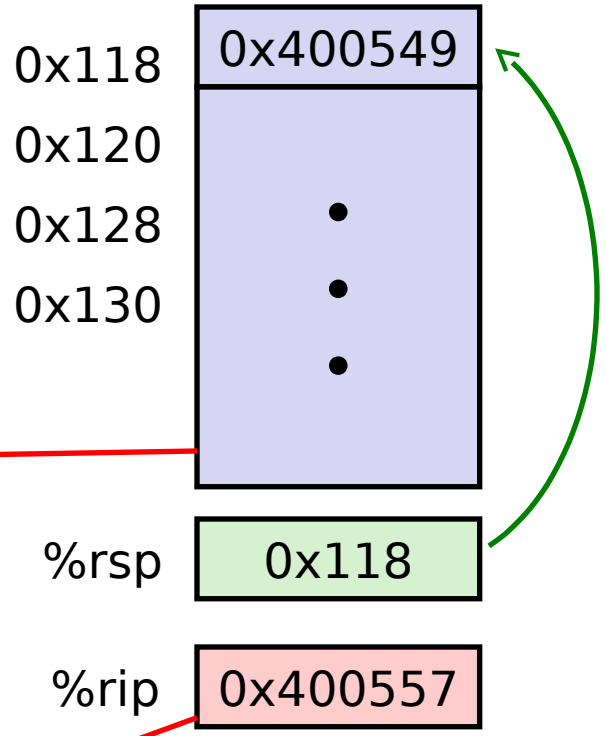
```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



Control Flow Example #3

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

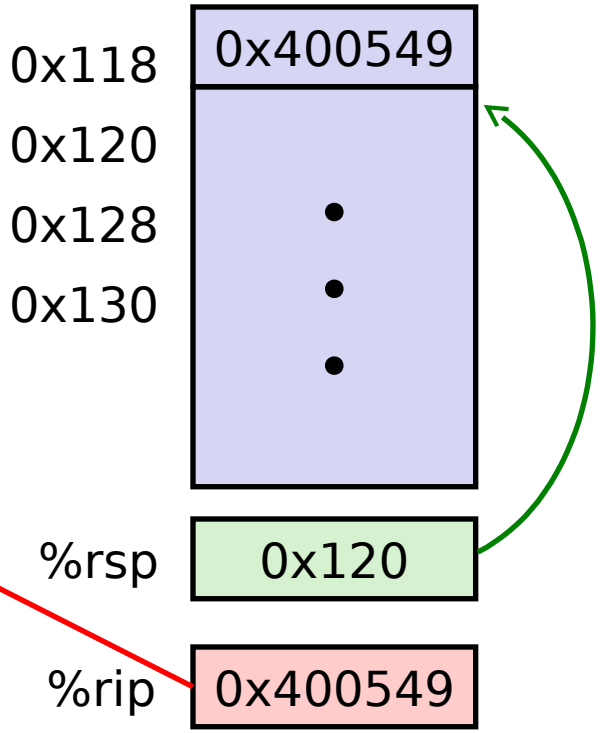
```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



Control Flow Example #4

```
0000000000400540 <multstore>:
  •
  •
  400544: callq  400550 <mult2>
  400549: mov    %rax,(%rbx)
  •
  •
```

```
0000000000400550 <mult2>:
  400550:  mov    %rdi,%rax
  •
  •
  400557:  retq
```



Procedure Data Flow

Registers

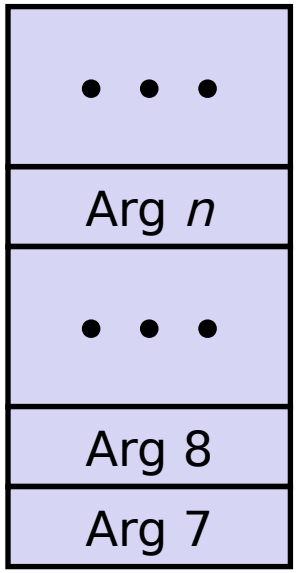
- **First 6 arguments**

%rdi
%rsi
%rdx
%rcx
%r8
%r9

- **Return value**

%rax

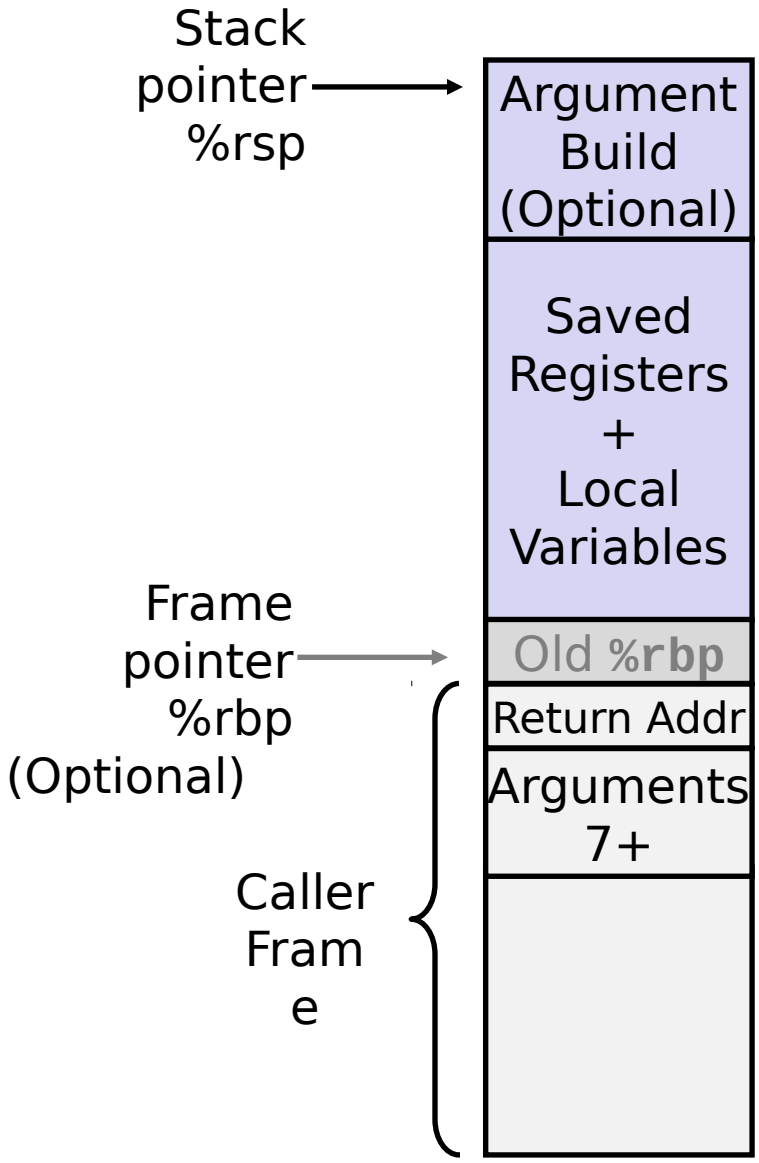
Stack



- **Only allocate stack space when needed**

x86-64/Linux Stack Frame

- **Current Stack Frame ("Top" to Bottom)**
  - "Argument build:" Parameters for function about to call
  - Local variables If can't keep in registers
  - Saved register context
  - Old frame pointer
- **Caller Stack Frame**
  - Return address
    - Pushed by call instruction
  - Arguments for this call



Register Saving Conventions

- **When procedure yoo calls who:**
  - yoo is the caller
  - who is the callee
- **Can register be used for temporary storage?**

```
yoo:
    • • •
    movq $15213, %rdx
    call who
    addq %rdx, %rax
    • • •
    ret
```

```
who:
    • • •
    subq $18213, %rdx
    • • •
    ret
```

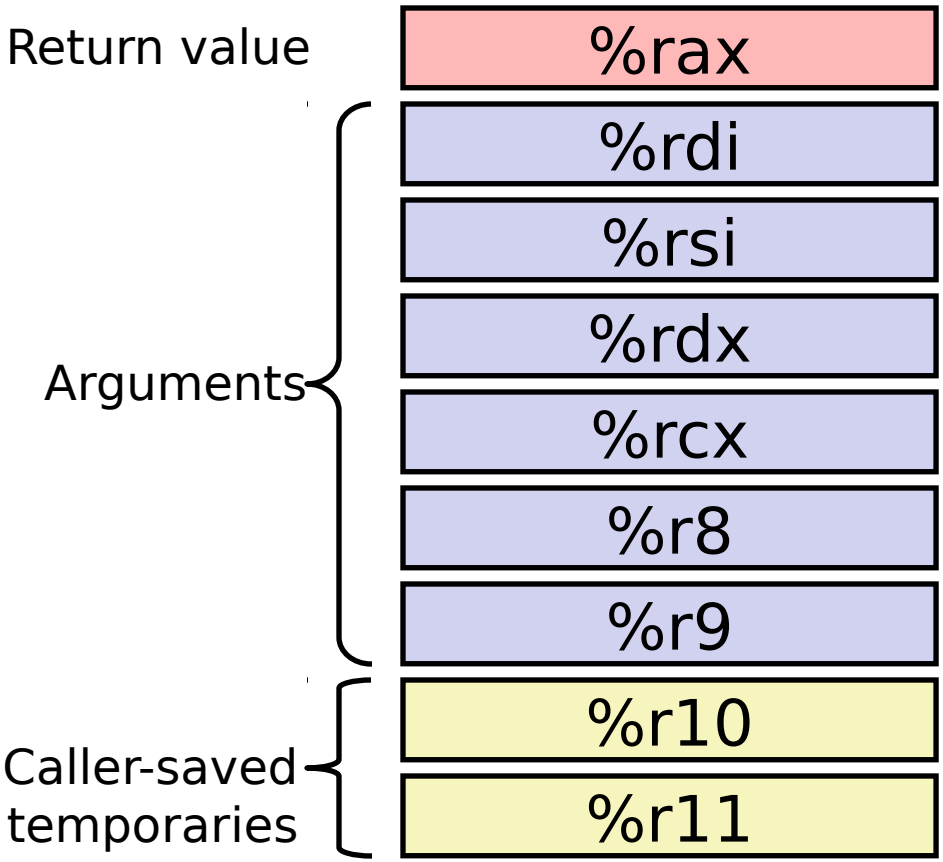
  - Contents of register %rdx overwritten by who
  - This could be trouble → something should be done!
    - Need some coordination

Register Saving Conventions

- **When procedure yoo calls who:**
  - yoo is the caller
  - who is the callee
- **Can register be used for temporary storage?**
- **Conventions**
  - “Caller Saved”
    - Caller saves temporary values in its frame before the call
  - “Callee Saved”
    - Callee saves temporary values in its frame before using
    - Callee restores them before returning to caller

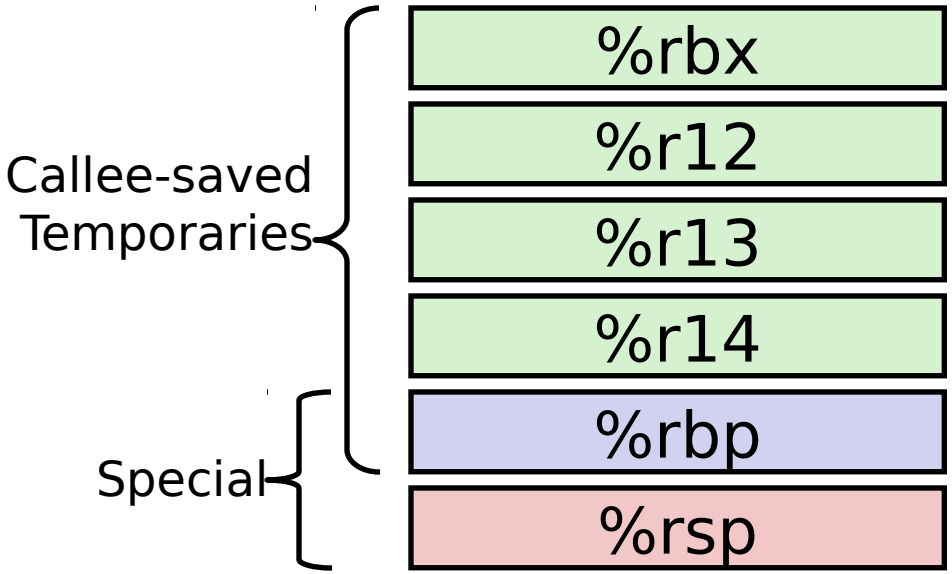
x86-64 Linux Register Usage #1

- **%rax**
  - Return value
  - Also caller-saved
  - Can be modified by procedure
- **%rdi**, …, **%r9**
  - Arguments
  - Also caller-saved
  - Can be modified by procedure
- **%r10**, **%r11**
  - Caller-saved
  - Can be modified by procedure



x86-64 Linux Register Usage #2

- **%rbx, %r12, %r13, %r14**
  - Callee-saved
  - Callee must save & restore
- **%rbp**
  - Callee-saved
  - Callee must save & restore
  - May be used as frame pointer
  - Can mix & match
- **%rsp**
  - Special form of callee save
  - Restored to original value upon exit from procedure



Example 4:

The disassembled code for two functions `first` and `last` is shown below, along with the code for a call of `first` by function `main`:

```
     Disassembly of last(long u, long v)
     u in %rdi, v in %rsi
1    0000000000400540 <last>:
2      400540:  48 89 f8              mov    %rdi,%rax          L1: u
3      400543:  48 0f af c6           imul   %rsi,%rax          L2: u*v
4      400547:  c3                    retq                      L3: Return

     Disassembly of last(long x)
     x in %rdi
5    0000000000400548 <first>:
6      400548:  48 8d 77 01           lea    0x1(%rdi),%rsi     F1: x+1
7      40054c:  48 83 ef 01           sub    $0x1,%rdi          F2: x-1
8      400550:  e8 eb ff ff ff        callq  400540 <last>      F3: Call last(x-1,x+1)
9      400555:  f3 c3                 repz retq                 F4: Return
         .
         .
         .
10     400560:  e8 e3 ff ff ff        callq  400548 <first>     M1: Call first(10)
11     400565:  48 89 c2              mov    %rax,%rdx          M2: Resume
```

Each of these instructions is given a label, similar to those in Figure 3.27(a). Starting with the calling of `first(10)` by `main`, fill in the following table to trace instruction execution through to the point where the program returns back to `main`.

```
     Disassembly of last(long u, long v)
     u in %rdi, v in %rsi
1    0000000000400540 <last>:
2      400540:  48 89 f8              mov    %rdi,%rax        L1: u
3      400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4      400547:  c3                    retq                    L3: Return

     Disassembly of last(long x)
     x in %rdi
5    0000000000400548 <first>:
6      400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7      40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8      400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9      400555:  f3 c3                 repz retq               F4: Return
                .
                .
                .
10     400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11     400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1								
F2								
F3								
L1								
L2								
L3								
F4								
M2								

```
     Disassembly of last(long u, long v)
     u in %rdi, v in %rsi
1    0000000000400540 <last>:
2      400540:  48 89 f8              mov    %rdi,%rax        L1: u
3      400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4      400547:  c3                    retq                    L3: Return

     Disassembly of last(long x)
     x in %rdi
5    0000000000400548 <first>:
6      400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7      40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8      400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9      400555:  f3 c3                 repz retq               F4: Return
                .
                .
                .
10     400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11     400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2								
F3								
L1								
L2								
L3								
F4								
M2								

```
     Disassembly of last(long u, long v)
     u in %rdi, v in %rsi
1    0000000000400540 <last>:
2      400540:  48 89 f8              mov    %rdi,%rax        L1: u
3      400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4      400547:  c3                    retq                    L3: Return

     Disassembly of last(long x)
     x in %rdi
5    0000000000400548 <first>:
6      400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7      40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8      400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9      400555:  f3 c3                 repz retq               F4: Return
       .
       .
       .
10     400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11     400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3								
L1								
L2								
L3								
F4								
M2								

```
                Disassembly of last(long u, long v)
                u in %rdi, v in %rsi
1   0000000000400540 <last>:
2     400540:  48 89 f8              mov    %rdi,%rax        L1: u
3     400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4     400547:  c3                    retq                    L3: Return

                Disassembly of last(long x)
                x in %rdi
5   0000000000400548 <first>:
6     400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7     40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8     400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9     400555:  f3 c3                 repz retq               F4: Return
                .
                .
                .
10    400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11    400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

	Instruction		State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1								
L2								
L3								
F4								
M2								

```
    Disassembly of last(long u, long v)
    u in %rdi, v in %rsi
1   0000000000400540 <last>:
2     400540:  48 89 f8          mov   %rdi,%rax       L1: u
3     400543:  48 0f af c6       imul  %rsi,%rax       L2: u*v
4     400547:  c3                retq                  L3: Return

    Disassembly of last(long x)
    x in %rdi
5   0000000000400548 <first>:
6     400548:  48 8d 77 01       lea   0x1(%rdi),%rsi  F1: x+1
7     40054c:  48 83 ef 01       sub   $0x1,%rdi       F2: x-1
8     400550:  e8 eb ff ff ff    callq 400540 <last>   F3: Call last(x-1,x+1)
9     400555:  f3 c3             repz retq             F4: Return
      .
      .
      .
10    400560:  e8 e3 ff ff ff    callq 400548 <first>  M1: Call first(10)
11    400565:  48 89 c2          mov   %rax,%rdx       M2: Resume
```

	Instruction		State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1	0X400540	mov	9	11	--	0x7fffffffe810	0X400555	Call last(9,11)
L2								
L3								
F4								
M2								

```
     Disassembly of last(long u, long v)
     u in %rdi, v in %rsi
1    0000000000400540 <last>:
2      400540:  48 89 f8          mov   %rdi,%rax       L1: u
3      400543:  48 0f af c6       imul  %rsi,%rax       L2: u*v
4      400547:  c3                retq                  L3: Return

     Disassembly of last(long x)
     x in %rdi
5    0000000000400548 <first>:
6      400548:  48 8d 77 01       lea   0x1(%rdi),%rsi  F1: x+1
7      40054c:  48 83 ef 01       sub   $0x1,%rdi       F2: x-1
8      400550:  e8 eb ff ff ff    callq 400540 <last>   F3: Call last(x-1,x+1)
9      400555:  f3 c3             repz retq             F4: Return
       .
       .
       .
10     400560:  e8 e3 ff ff ff    callq 400548 <first>  M1: Call first(10)
11     400565:  48 89 c2          mov   %rax,%rdx       M2: Resume
```

	Instruction		State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1	0X400540	mov	9	11	--	0x7fffffffe810	0X400555	Call last(9,11)
L2	0X400543	imul	9	11	9	0x7fffffffe810	0X400555	Cont. last
L3								
F4								
M2								

```
    Disassembly of last(long u, long v)
    u in %rdi, v in %rsi
1   0000000000400540 <last>:
2     400540:  48 89 f8          mov   %rdi,%rax       L1: u
3     400543:  48 0f af c6       imul  %rsi,%rax       L2: u*v
4     400547:  c3                retq                  L3: Return

    Disassembly of last(long x)
    x in %rdi
5   0000000000400548 <first>:
6     400548:  48 8d 77 01       lea   0x1(%rdi),%rsi  F1: x+1
7     40054c:  48 83 ef 01       sub   $0x1,%rdi       F2: x-1
8     400550:  e8 eb ff ff ff    callq 400540 <last>   F3: Call last(x-1,x+1)
9     400555:  f3 c3             repz retq             F4: Return
        .
        .
        .
10    400560:  e8 e3 ff ff ff    callq 400548 <first>  M1: Call first(10)
11    400565:  48 89 c2          mov   %rax,%rdx       M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1	0X400540	mov	9	11	--	0x7fffffffe810	0X400555	Call last(9,11)
L2	0X400543	imul	9	11	9	0x7fffffffe810	0X400555	Cont. last
L3	0X400547	retq	9	11	99	0x7fffffffe810	0X400555	u = u * v
F4								
M2								

```
    Disassembly of last(long u, long v)
    u in %rdi, v in %rsi
1   0000000000400540 <last>:
2     400540:  48 89 f8          mov   %rdi,%rax       L1: u
3     400543:  48 0f af c6       imul  %rsi,%rax       L2: u*v
4     400547:  c3                retq                  L3: Return

    Disassembly of last(long x)
    x in %rdi
5   0000000000400548 <first>:
6     400548:  48 8d 77 01       lea   0x1(%rdi),%rsi  F1: x+1
7     40054c:  48 83 ef 01       sub   $0x1,%rdi       F2: x-1
8     400550:  e8 eb ff ff ff    callq 400540 <last>   F3: Call last(x-1,x+1)
9     400555:  f3 c3             repz retq             F4: Return
      .
      .
      .
10    400560:  e8 e3 ff ff ff    callq 400548 <first>  M1: Call first(10)
11    400565:  48 89 c2          mov   %rax,%rdx       M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1	0X400540	mov	9	11	--	0x7fffffffe810	0X400555	Call last(9,11)
L2	0X400543	imul	9	11	9	0x7fffffffe810	0X400555	Cont. last
L3	0X400547	retq	9	11	99	0x7fffffffe810	0X400555	u = u * v
F4	0X400555	retq	9	11	99	0x7fffffffe818	0X400565	Return to first
M2								

```
     Disassembly of last(long u, long v)
     u in %rdi, v in %rsi
1    0000000000400540 <last>:
2      400540:  48 89 f8              mov    %rdi,%rax        L1: u
3      400543:  48 0f af c6           imul   %rsi,%rax        L2: u*v
4      400547:  c3                    retq                    L3: Return

     Disassembly of last(long x)
     x in %rdi
5    0000000000400548 <first>:
6      400548:  48 8d 77 01           lea    0x1(%rdi),%rsi   F1: x+1
7      40054c:  48 83 ef 01           sub    $0x1,%rdi        F2: x-1
8      400550:  e8 eb ff ff ff        callq  400540 <last>    F3: Call last(x-1,x+1)
9      400555:  f3 c3                 repz retq               F4: Return
       .
       .
       .
10     400560:  e8 e3 ff ff ff        callq  400548 <first>   M1: Call first(10)
11     400565:  48 89 c2              mov    %rax,%rdx        M2: Resume
```

Instruction			State values (at beginning)					
Label	PC	Instruction	%rdi	%rsi	%rax	%rsp	*%rsp	Description
M1	0x400560	callq	10	—	—	0x7fffffffe820	—	Call first(10)
F1	0X400548	lea	10	--	--	0x7fffffffe818	0X400565	Entry of first
F2	0X40054c	sub	10	11	--	0x7fffffffe818	0X400565	Cont. first
F3	0X400550	callq	9	11	--	0x7fffffffe818	0X400565	x = x - 1
L1	0X400540	mov	9	11	--	0x7fffffffe810	0X400555	Call last(9,11)
L2	0X400543	imul	9	11	9	0x7fffffffe810	0X400555	Cont. last
L3	0X400547	retq	9	11	99	0x7fffffffe810	0X400555	u = u * v
F4	0X400555	retq	9	11	99	0x7fffffffe818	0X400565	Return to first
M2	0X400565	mov	9	11	99	0x7fffffffe820	--	Return to main

Example 5:

A C function `procprob` has four arguments `u`, `a`, `v`, and `b`. Each is either a signed number or a pointer to a signed number, where the numbers have different sizes. The function has the following body:

```
*u += a;
*v += b;
return sizeof(a) + sizeof(b);
```

It compiles to the following x86-64 code:

```
1  procprob:
2    movslq  %edi, %rdi
3    addq    %rdi, (%rdx)
4    addb    %sil, (%rcx)
5    movl    $6, %eax
6    ret
```

movslq: move sign-extended long word to quad word

%rdi → 1. argument
%rsi → 2. argument
%rcx → 3. argument
%rdx → 4. argument

Determine a valid ordering and types of the four parameters. There are two correct answers.

Example 5:

A C function `procprob` has four arguments `u`, `a`, `v`, and `b`. Each is either a signed number or a pointer to a signed number, where the numbers have different sizes. The function has the following body:

```
*u += a;
*v += b;
return sizeof(a) + sizeof(b);
```

It compiles to the following x86-64 code:

```
1  procprob:
2    movslq  %edi, %rdi
3    addq    %rdi, (%rdx)
4    addb    %sil, (%rcx)
5    movl    $6, %eax
6    ret
```

movslq: move sign-extended long word to quad word

%rdi → 1. argument
%rsi → 2. argument
%rdx → 3. argument
%rcx → 4. argument

a: extended from l to q (int). in %rdi (1st)
u: 8 byte value is put where it points (long*). in %rdx (3rd)
v: 1 byte value is put where it points (char*). in %rcx (4th)
b: sizeof(a) + sizeof(b) = 2 (short). in %rsi (2nd)

```
(int a, short b, long *u, char *v)
```

Example 6:

Consider a function P, which generates local values, named a0–a8. It then calls function Q using these generated values as arguments. Gcc produces the following code for the first part of P:

```
    long P(long x)
    x in %rdi
1   P:
2     pushq   %r15
3     pushq   %r14
4     pushq   %r13
5     pushq   %r12
6     pushq   %rbp
7     pushq   %rbx
8     subq    $24, %rsp
9     movq    %rdi, %rbx
10    leaq    1(%rdi), %r15
11    leaq    2(%rdi), %r14
12    leaq    3(%rdi), %r13
13    leaq    4(%rdi), %r12
14    leaq    5(%rdi), %rbp
15    leaq    6(%rdi), %rax
16    movq    %rax, (%rsp)
17    leaq    7(%rdi), %rdx
18    movq    %rdx, 8(%rsp)
19    movl    $0, %eax
20    call    Q
```

A. Identify which local values get stored in callee-saved registers.

B. Identify which local values get stored on the stack.

C. Explain why the program could not store all of the local values in callee-saved registers.

Example 6:

A. Identify which local values get stored in callee-saved registers.

```
    long P(long x)
    x in %rdi
1   P:
2     pushq   %r15
3     pushq   %r14
4     pushq   %r13
5     pushq   %r12
6     pushq   %rbp
7     pushq   %rbx
8     subq    $24, %rsp
9     movq    %rdi, %rbx
10    leaq    1(%rdi), %r15
11    leaq    2(%rdi), %r14
12    leaq    3(%rdi), %r13
13    leaq    4(%rdi), %r12
14    leaq    5(%rdi), %rbp
15    leaq    6(%rdi), %rax
16    movq    %rax, (%rsp)
17    leaq    7(%rdi), %rdx
18    movq    %rdx, 8(%rsp)
19    movl    $0, %eax
20    call    Q
```

Local values a0-a5 are saved in registers %rbx, %r15, %r14, %r13, %r12, %rbp (lines 9-14).

Example 6:

B. Identify which local values get stored on the stack.

```
    long P(long x)
    x in %rdi
1   P:
2     pushq   %r15
3     pushq   %r14
4     pushq   %r13
5     pushq   %r12
6     pushq   %rbp
7     pushq   %rbx
8     subq    $24, %rsp
9     movq    %rdi, %rbx
10    leaq    1(%rdi), %r15
11    leaq    2(%rdi), %r14
12    leaq    3(%rdi), %r13
13    leaq    4(%rdi), %r12
14    leaq    5(%rdi), %rbp
15    leaq    6(%rdi), %rax
16    movq    %rax, (%rsp)
17    leaq    7(%rdi), %rdx
18    movq    %rdx, 8(%rsp)
19    movl    $0, %eax
20    call    Q
```

Local values a6 and a7 are stored in stack at offsets 0 and 8 relative to the stack pointer (lines 16 and 18)

Example 6:

C. Explain why the program could not store all of the local values in callee-saved registers.

```
    long P(long x)
    x in %rdi
1   P:
2     pushq   %r15
3     pushq   %r14
4     pushq   %r13
5     pushq   %r12
6     pushq   %rbp
7     pushq   %rbx
8     subq    $24, %rsp
9     movq    %rdi, %rbx
10    leaq    1(%rdi), %r15
11    leaq    2(%rdi), %r14
12    leaq    3(%rdi), %r13
13    leaq    4(%rdi), %r12
14    leaq    5(%rdi), %rbp
15    leaq    6(%rdi), %rax
16    movq    %rax, (%rsp)
17    leaq    7(%rdi), %rdx
18    movq    %rdx, 8(%rsp)
19    movl    $0, %eax
20    call    Q
```

6 callee saved registers are used for a0-a5. Rest had to be stored in stack.